

Finite-sample learning dynamics of a shared measurement representation

why item sharpness is recovered slowly, and how a separate code fixes it

Wenrui Yuan

The model and the three numbers

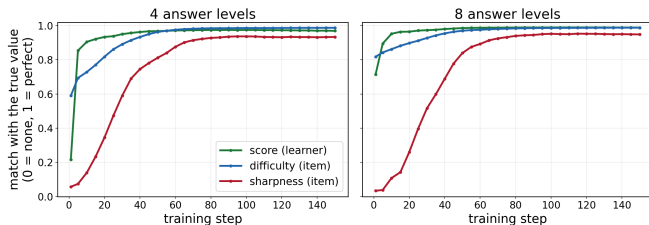
A sequence model reads a learner's answer history and predicts the next answer. To do that it works out three numbers through one shared internal representation.

- **score**, how strong a learner is (read from the encoder state).
- **difficulty**, how hard a question is (read from the item code).
- **sharpness**, how clearly a question separates strong from weak learners (also read from the item code).

We never supply these numbers and never add a target for them. The model is trained only to predict answers. On simulated data the true numbers are known, so we measure recovery as **match in order** (rank, 0 to 1), which is the only scale-free thing to ask of this model. We study the **finite-sample** regime, realistic data and a realistic training budget, because that is where real models live.

The phenomenon, sharpness is the hard one

Score and difficulty are learned fast; sharpness lags



Score and difficulty recover fast. Sharpness is slow and unreliable, because the prediction barely depends on it, sharpness only matters for learners whose strength sits near a question's difficulty, and washes out elsewhere.

Sharpness and difficulty are both read from the same shared item code. So the question is what that sharing does to the hard number while the model trains.

Three explanations, only one is interesting

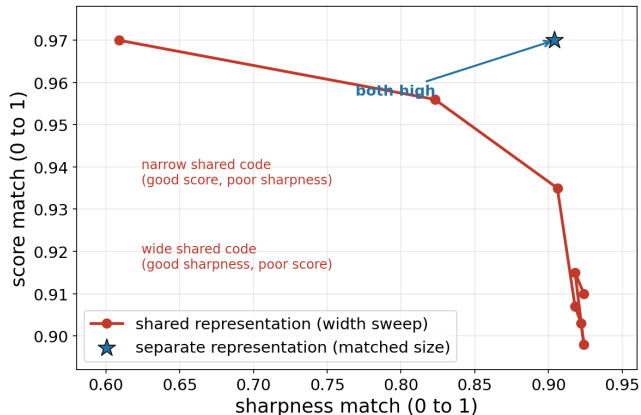
A shared item code must serve both difficulty and sharpness. Sharpness recovers poorly under sharing. Why?

- ① **Capacity.** The shared code is simply too small. A wider one would fix it. (Trivial.)
- ② **Expressivity.** Even an arbitrarily wide shared code cannot hold the score-optimal and sharpness-optimal directions at once. (A property of the model, not of training.)
- ③ **Dynamics.** A shared code with room for both is still driven by training toward the score solution, leaving sharpness behind. (A finite-sample learning-dynamics effect.)

The study is a sequence of tests, each designed to kill one explanation. The control on each is stated on its slide.

Test 1, it is allocation, not budget

One shared code cannot get both
separate code sits above the curve

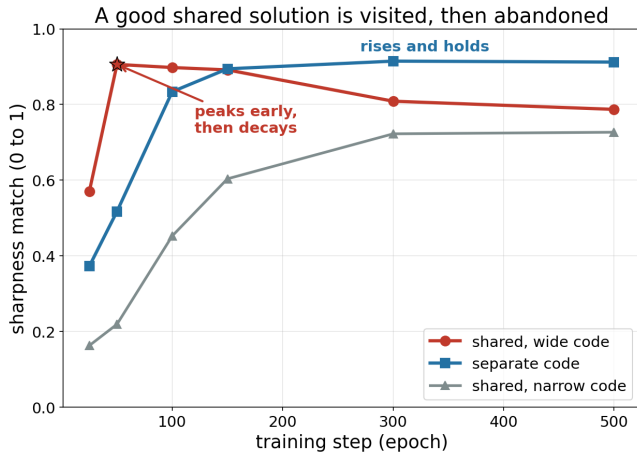


Design. Sweep the shared code from narrow to wide, recording both score and sharpness recovery. Place a model that gives sharpness its **own separate code** on the same axes, at **matched total size** (the control, so this is about allocation, not budget).

Result. The shared code traces a trade-off, narrow gives good score and poor sharpness, wide gives the reverse, and no width gets both. The separate code, at the same total size, sits above the curve, score and sharpness both high.

Conclusion. Capacity is eliminated. The problem is how the code is allocated, not how big it is.

Test 2, the good solution is visited, then abandoned

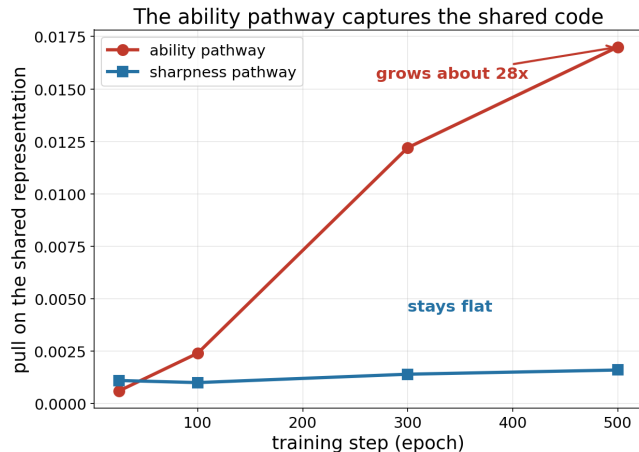


Design. Checkpoint training and recover sharpness at each step, for the wide shared code, the separate code, and the narrow shared code (3 seeds).

Result. The wide shared code reaches high sharpness early (about epoch 50, with score also high there), then **decays** with more training, from 0.91 at epoch 50 to 0.79 by epoch 500. The separate code rises and **holds**.

Conclusion. Expressivity is eliminated. The good solution is reachable and is actually reached, then training leaves it. That is a dynamics effect, not a ceiling.

Test 2, the mechanism, the code is captured



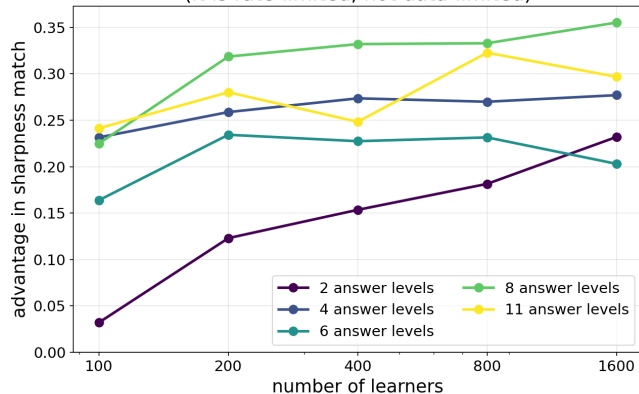
Design. Split the pull on the shared code into the ability pathway and the sharpness pathway, and track each over training (one instrumented run).

Result. The ability pathway's pull grows about **28 times** over training while the sharpness pathway's stays flat. The two are roughly perpendicular, so it is not a tug of war. The shared code's learning is lopsided, the ability direction is far steeper to descend than the sharpness direction, so it is resolved first and the slow sharpness direction last.

Conclusion. The fast ability direction dominates the shared code as training proceeds, so the early good sharpness solution is left behind. This is the same lopsidedness that sets the size of the effect (next).

Test 3, more data does not fix it

More data does not shrink the advantage
(it is rate-limited, not data-limited)

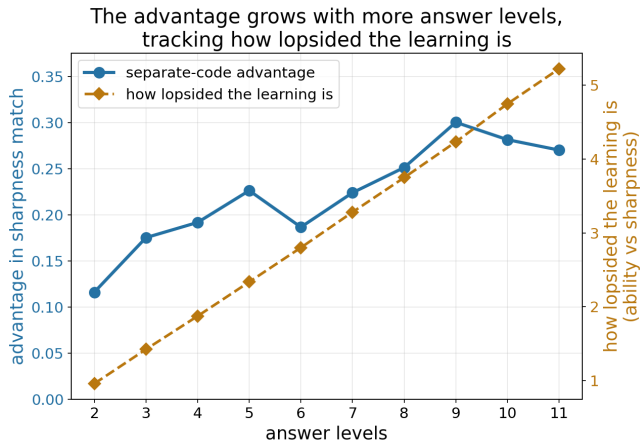


Design. Vary the number of learners at a fixed training budget, and measure the separate-code advantage on sharpness, across answer-level counts.

Result. The advantage does **not** shrink with more data. It holds or widens, from about +0.03 to +0.23 at two answer levels as learners grow from 100 to 1600, and it sits higher for richer scales.

Conclusion. This is the finite-sample signature. The effect is **rate-limited**, not data-limited. More learners under the same budget leave the slow sharpness mode further behind, while the separate code keeps up.

Test 3, and it scales with the task

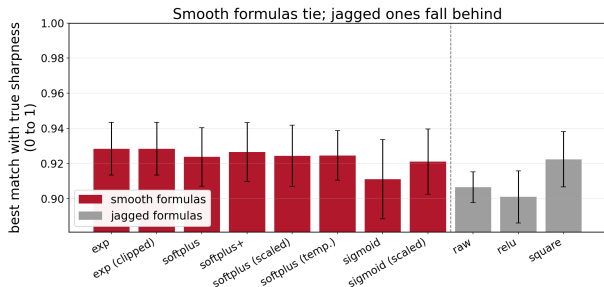


Design. Sweep the number of answer levels, and compare the separate-code advantage to how lopsided the learning speeds are between the ability and sharpness directions (a quantity we can compute in closed form).

Result. The advantage grows with answer levels and **tracks the lopsidedness** closely (rank agreement 0.89 across two to eleven levels). Richer grading scales make the learning more lopsided, and the separate code helps more.

Conclusion. The size of the effect is governed by how unevenly the two numbers are learned, which the task sets.

A side question we tested and ruled out



Sharpness must stay positive, so it is read through an always-positive formula. A natural guess was that one formula, the exponential, is what makes sharpness learnable, by rescaling the training updates.

Tested fairly (matched starting point, per-formula tuning), all smooth formulas tie, only crude ones lag. A **direct test** of the update-rescaling idea also came up empty.

Conclusion. The formula is not the lever, this hypothesis is ruled out. What matters is the representation, the separate code, exactly as Tests 1 to 3 show.

Why, and why it is a rate effect

The tests share one mechanism, set out in the supporting analysis.

- Sharpness is the low-information number, the prediction's sensitivity to it is the learner-question gap times the answer uncertainty, so it vanishes when the gap is zero (a learner right at the question's level) and again far away.
- In a shared code, the high-information ability direction is learned fast and captures the code, so the low-information sharpness direction resolves last. The lag is set by how lopsided the two are, which grows with answer levels.
- This sets the **rate** of recovery, not the endpoint. With unlimited training the shared and separate codes reach the same place, so the separate code is a finite-budget speed and reliability gain, on the order of sharpness, not its size.

So the effect is real and explained, and it is a finite-sample, rate-limited, representation phenomenon, not a property of the final optimum.

Takeaway

One line

In finite samples, a shared representation is captured by the easy-to-learn number during training, so the hard one is recovered slowly. Giving the hard one its own code holds the good solution.

- Not capacity (allocation, at matched size), not expressivity (the good solution is reached then left), but learning dynamics (the code is captured, 28x).
- The fix is a finite-sample, rate-limited gain that grows with the task and does not wash out with more data. It recovers the order of sharpness, not its size.
- A wrong side hypothesis (the positive formula) was ruled out, it is the representation, not the formula.

Why it matters. If the recovered numbers are artifacts of how the representation is allocated and trained, a stable measurement scale is not guaranteed at a finite budget. Separating the representation is a validity condition for it. **Scope and next.** Simulated data, three to ten repeats. Next, the full per-number channel grid, the nominal-response head, and real learner data.